

Sistema de gestión Triverde

Documento de Instalación — Incremento 1

Versión 1.0 · Ingeniería de Software, Universidad Andrés Bello · Agosto 2026

Este documento describe cómo instalar y poner en marcha el sistema de gestión operativa de Triverde en su **Incremento 1**. El sistema tiene dos componentes:

- **Backend** — Django + Django REST Framework sobre PostgreSQL, empaquetado en Docker Compose.
- **Vistas (frontend)** — aplicación web instalable (PWA) construida con React + Vite.

La vía recomendada levanta el backend con **Docker Compose** (una sola orden migra la base de datos, siembra los datos mínimos y arranca el servidor) y las Vistas con **Node.js**. Al final se indica una vía alternativa sin Docker.

1. Requisitos previos

Componente	Requisito	Comprobación
Backend (recomendado)	Docker Desktop en ejecución	<code>docker --version</code>
Vistas (frontend)	Node.js 20 LTS o superior y <code>npm</code>	<code>node --version</code>
Backend (vía alternativa)	Python 3.13 y <code>pip</code>	<code>python --version</code>

Puertos usados en local: **8000** (backend) · **5432** (PostgreSQL) · **5173** (servidor de desarrollo de las Vistas). Deben estar libres.

2. Obtener el código

Clonar el repositorio y situarse en la carpeta del proyecto:

```
git clone https://github.com/kenjikaese/Triverde.git
cd Triverde
```

La estructura relevante para la instalación es:

```
Triverde/
  backend/    Django + DRF + PostgreSQL (Docker Compose)
  frontend/   PWA (React + Vite)
```

3. Backend (vía recomendada: Docker Compose)

Con Docker Desktop en ejecución:

```
cd backend
docker compose up -d --build
```

Esta orden levanta PostgreSQL, espera a que esté sano, aplica las migraciones, siembra los datos iniciales (roles, usuario administrador, parámetros y tarifas base) y arranca el servidor con Gunicorn.

Cuando termine, el backend queda disponible en:

- **Panel de administración:** `http://localhost:8000/admin/`
- **API REST:** `http://localhost:8000/api/v1/`

Credenciales iniciales

Usuario	Clave	Rol
admin	admin	Administrador

Importante (seguridad). El usuario `admin/admin` es solo para la primera puesta en marcha. Debe cambiarse la contraseña de inmediato desde el panel de administración o creando un usuario nuevo.

Detener y limpiar

```
docker compose down      # detiene los contenedores
docker compose down -v   # además borra la base de datos (volumen)
```

4. Vistas (frontend)

Desde otra terminal, en la raíz del proyecto:

```
cd frontend
npm install
```

Modo desarrollo

```
npm run dev
```

Las Vistas quedan en `http://localhost:5173`. En desarrollo, las peticiones a `/api` se redirigen automáticamente al backend en `http://localhost:8000` (proxy de Vite), por lo que **no** hay que configurar nada más si el backend está corriendo según el paso 3.

Construcción para producción

```
npm run build
```

Genera los archivos estáticos en `frontend/dist/`, listos para servir. Para que apunten a un backend distinto del local, definir la variable `VITE_API_URL` antes de construir (copiar `frontend/.env.example` a `frontend/.env` y fijar el origen del backend).

5. Verificación de la instalación

1. **Backend arriba:** abrir `http://localhost:8000/admin/` e iniciar sesión con `admin` / `admin`. Deben verse los módulos de Acceso, Configuración, Mantenedores y Recepción.
2. **API responde:** obtener un token de acceso:

```
curl -X POST http://localhost:8000/api/v1/auth/login/ \
-H "Content-Type: application/json" \
-d '{"username": "admin", "password": "admin"}'
```

Debe devolver un `token` y los datos del usuario.

3. **Vistas arriba:** abrir `http://localhost:5173`, iniciar sesión y verificar que carga el panel de control.
4. **Pruebas automatizadas (opcional):** con el backend levantado en Docker,

```
docker compose exec web python manage.py test
```

Deben pasar las 30 pruebas del Incremento 1 (`OK`).

6. Vía alternativa: backend sin Docker

Si no se dispone de Docker, el backend puede correr con SQLite:

```
cd backend
python -m venv venv
# Windows (PowerShell): venv\Scripts\Activate.ps1
# Linux/Mac:            source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py seed_inicial
python manage.py runserver
```

Al no definir `DATABASE_URL`, el sistema usa una base de datos SQLite local (`backend/db.sqlite3`). El resto (admin, API, credenciales iniciales) funciona igual que en la vía con Docker.

7. Notas para el despliegue en servidor

Estas notas aplican al pasar del entorno local a un servidor real (universidad o VPS); no son necesarias para la instalación local del Incremento 1.

- **HTTPS obligatorio para la cámara.** La captura de fotografías del material (RF-19) requiere que el sistema se sirva bajo HTTPS; los navegadores bloquean la cámara en orígenes no seguros.
- **Variables de entorno del backend:** en producción deben fijarse `DJANGO_SECRET_KEY` (valor propio y secreto), `DJANGO_DEBUG=false` y `DJANGO_ALLOWED_HOSTS` con el dominio real. La `DATABASE_URL` debe apuntar a la base de datos PostgreSQL del servidor.
- **Origen de la API en las Vistas:** construir el frontend con `VITE_API_URL` apuntando al origen público del backend.
- El empaquetado en Docker Compose permite que el mismo despliegue corra igual en local, en el servidor de la universidad y en un VPS propio.